



Anatomy of a Task Instance

From Scheduled to Done



Cedrik Neumann

Cedrik Neumann

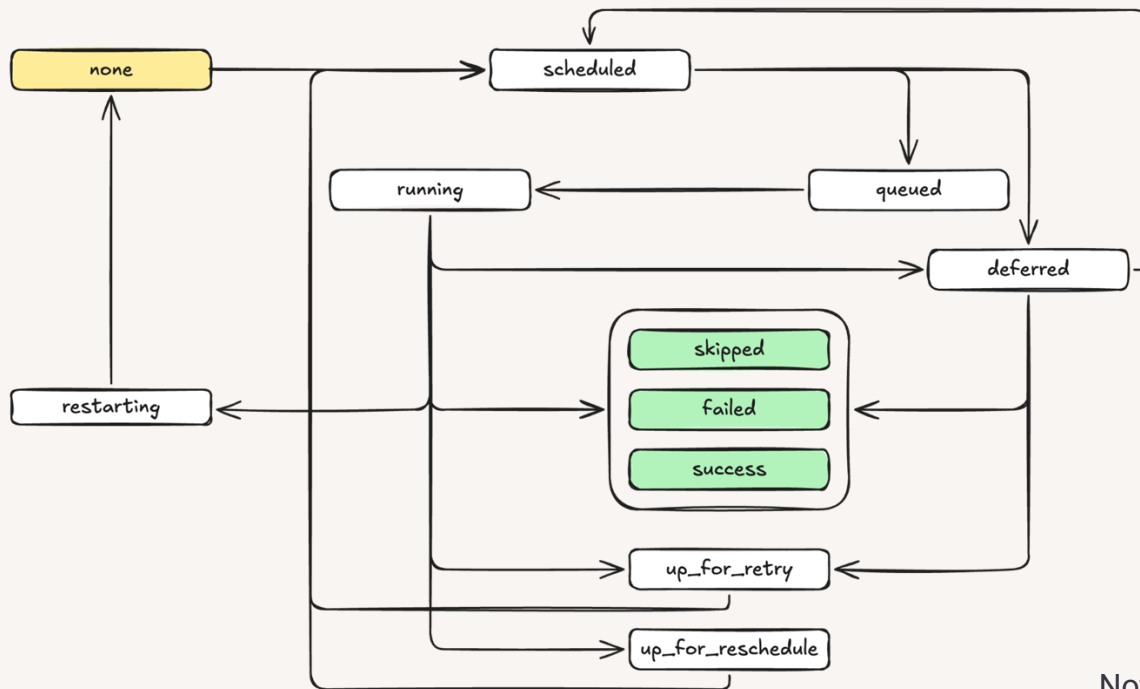
Solutions Architect

github.com/m1racoli

linkedin.com/in/cedrikneumann

ASTRONOMER

What happens in when and where?



Not included:

- removed
- upstream_failed
- awaiting_input

Different kinds of user code

What is user code?

Code that is not shipped with Airflow.

-  **DAG code** — written by the DAG author. **Fully untrusted**. Top-level Dag code, operators, callbacks, retry policies, triggers, ...
-  **Plugins** — generally untrusted, but deployment can be controlled by a trusted operator. Operator links, timetables, priority weight strategies, listeners, ...
-  **Configuration** — non Airflow internal, but usually controlled and maintained by trusted parties. Cluster policies, executors, secret backends, ...

The background of the slide features a repeating pattern of small, light gray triangles pointing downwards, arranged in a grid-like fashion across the top half of the image.

Step by step

ASTRONOMER

Phase 0: A new Dag Run

Dag processor

- `get_dagbag_import_timeout` is called
- Dag file is imported
- `task_policy` and `dag_policy` are called

Scheduler

- A timetable (or asset) decide whether a new dagrun (state=`queued`) is being created (considering `max_active_runs`)
- Task Instances (state=`none`) are created and `task_instance_mutation_hook` is called
- Dag Run is switched to `running` and `on_dag_run_running` is called

Phase 0: A new Dag Run



Phase 1: Schedule TIs ready to run

Scheduler

- Calculate `priority_weight` and materialize value in TI row
- Call `task_instance_mutation_hook`, this time with DagRun attached
- Evaluate trigger rules (dependencies fulfilled?)
- Potential terminal states: `skipped`, `upstream_failed`, `removed` or `none`
- If conditions are met, set TI to `scheduled`

Phase 1: Schedule TIs ready to run

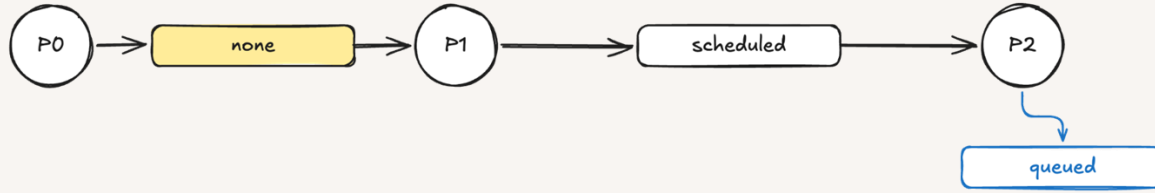


Phase 2: Queue TIs for execution

Scheduler

- Review scheduled TIs ordered by `priority_weight`, `logical_date`, `map_index`
- Consider capacity constraints (concurrency control): `pool_slots`, `max_active_tasks`, `max_active_tis_per_dag`, `max_active_tis_per_dagrun`, `parallelism`
- Set TIs satisfying the constraints to `queued` (or `deferred`)

Phase 2: Queue TIs for execution

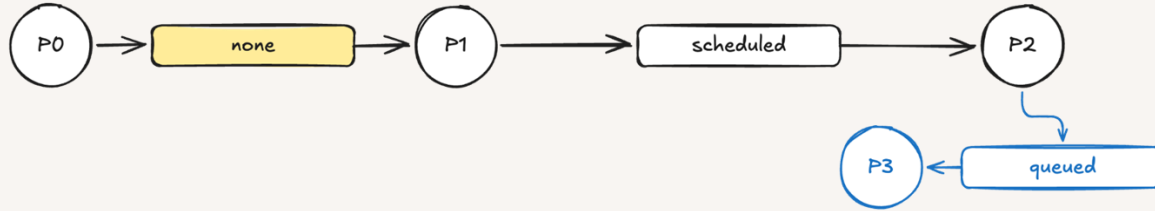


Phase 3: Executor handoff

Executor (Scheduler)

- No more scheduling decisions in core Airflow code
- The `Executor` implementation is responsible for acting upon queued tasks:
 - **Celery**: Push message to Celery queue
 - **Kubernetes**: Build pod from `pod_template` & `executor_config` and call `pod_mutation_hook`
 - **Local**: Run worker in forked process directly
 - **Edge**: Serve EdgeJob via API to Edge worker (in API server)

Phase 3: Executor handoff

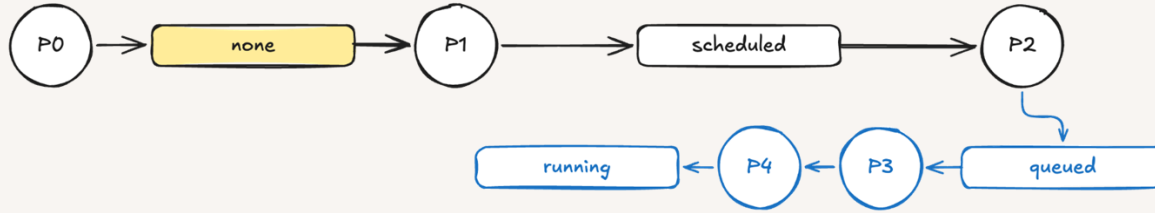


Phase 4: Start task supervision

Supervisor (Worker)

- Set up child task runner process in holding position
- Post TI state update to **running**
- Stop, if conflict during update (not **queued**)
- Kick off task runner
- Perform heartbeats
- Handle communication between task runner and API server
- Handle and upload logs from task runner

Phase 4: Start task supervision

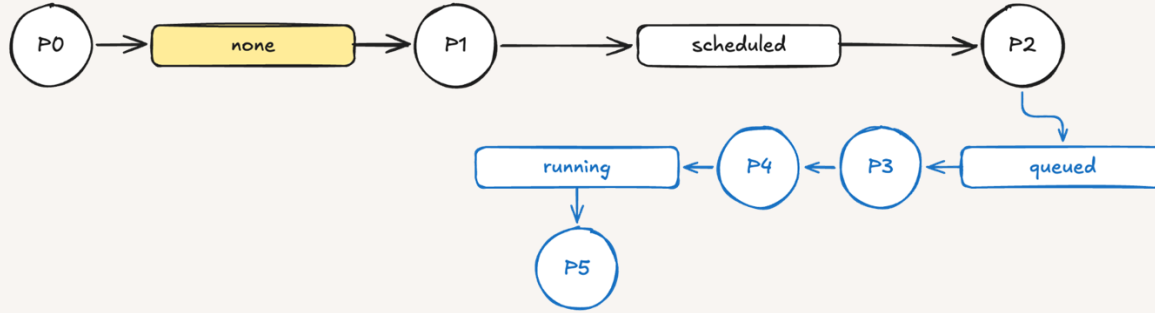


Phase 5: Execute the task

Task runner (Worker)

- Call `on_starting`
- Parse `Dag` file and `pod_mutation_hook` if task is `KubernetesPodOperator`
- Call `on_task_instance_running`
- Execute `task`
 - `pre_execute`
 - `on_execute_callback`
 - `execute`
 - `post_execute`

Phase 5: Execute the task

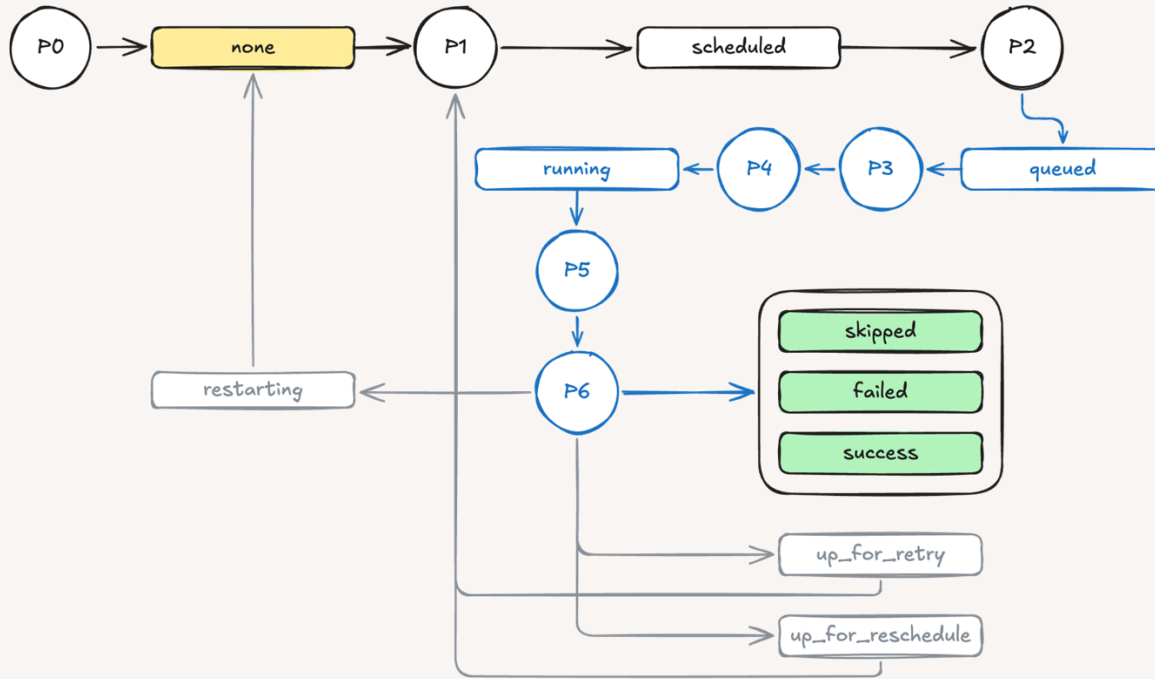


Phase 6: The other ways out

Task runner (worker)

- Decide TI state based on execution outcome:
 - Mark task as **success**
 - **Defer** task
 - Mark for **up_for_reschedule**
 - **Skip** task
 - Skip downstream tasks (**success**)
 - **Await input**
 - Evaluate **retry_policy** on failure (**up_for_retry**, **failed**)

Phase 6: The other ways out

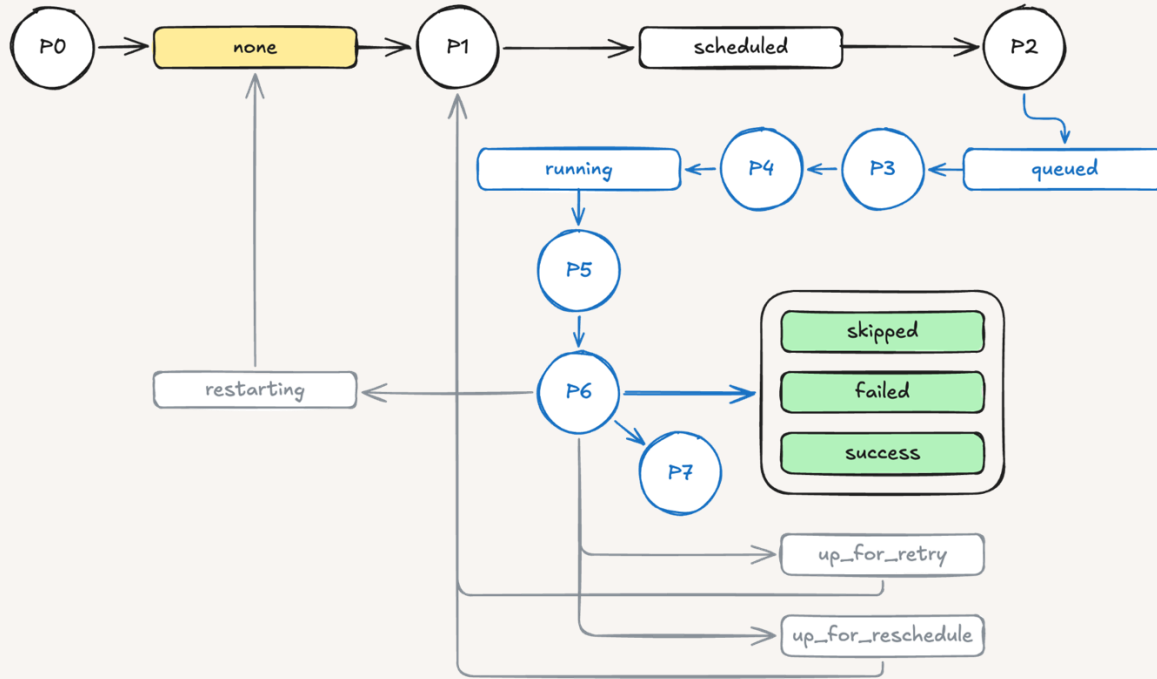


Phase 7: Overtime

Task runner (Worker)

- The task has completed and changed its state
- Report task duration
- Process `operator_extra_links`
- Run `task` callbacks
- Run `task` listeners
- Run `before_stopping` listener

Phase 7: Overtime

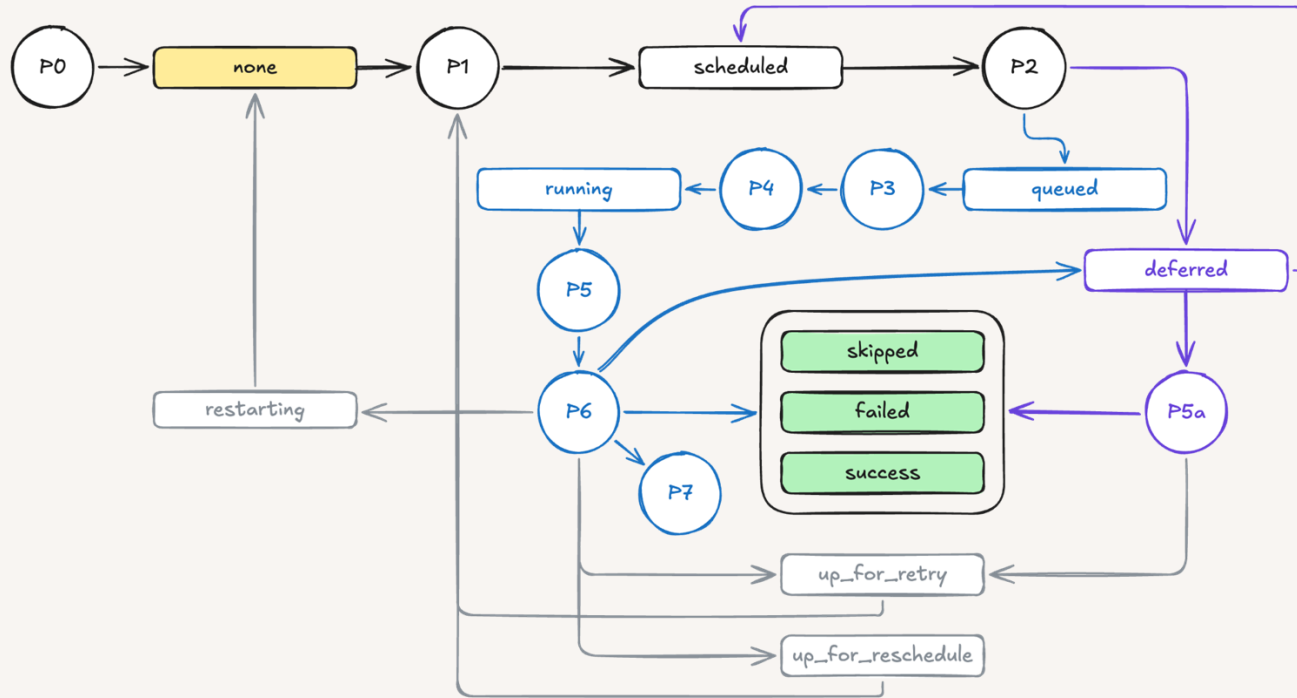


Phase 5a: Task deferral

Triggerer

- Task has been marked as **deferred** either by scheduler or in worker
- The task's  trigger is being run asynchronously
- Upon yielding a **TriggerEvent** the task returns to **scheduled**
- Failure ends up in **failed** or **up_for_retry**
- With **end_from_trigger** the task can also directly transition to **success** or **failed**

Phase 5a: Task deferral



Phase 8: Complete Dag Run

Scheduler

- Evaluate Dag Run completion and set it to **success**
- Call `on_dag_run_complete` Dag Run listeners
- Register **DagCallbackRequest** for completed Dag Runs
- Register **TaskCallbackRequest** for tasks without finalization

Dag processor

- Pick up **DagCallbackRequest** and call `on_dag_run_complete` Dag Run callbacks
- Pick up **TaskCallbackRequest** and call `on_task_complete` task callbacks



Recap

ASTRONOMER

Phases

0	A new Dag Run	Dag processor, Scheduler	  
1	Schedule TIs ready to run	Scheduler	 
2	Queue TIs for execution	Scheduler	
3	Executor handoff	Executor (Scheduler)	
4	Start task supervision	Supervisor (Worker)	
5	Execute the task	Task Runner (Worker)	  
5a	Deferral	Triggerer	 *
6	The other ways out	Task Runner (Worker)	
7	Overtime	Task Runner (Worker)	 
8	Afterwards	Scheduler, Dag processor	 

Callbacks

Name	Phase	Component
on_success_callback	7	Task runner (Worker)
on_failure_callback	7, 8	Task runner (Worker), Dag processor
on_retry_callback	7, 8	Task runner (Worker), Dag processor
on_execute_callback	5	Task runner (Worker)
on_skipped_callback	6	Task runner (Worker)
on_success_callback (DagRun)	8	Dag processor
on_failure_callback (DagRun)	8	Dag processor

Plugins

Name	Phase	Component
operator_extra_links	7	Task runner (worker)
global_operator_extra_links	-	API server
timetables	1	Scheduler
priority_weight_strategies	1	Scheduler
listeners	...	...

Listeners

Type	Name	Phase	Component
Lifecycle	on_starting	5	Task runner (worker)
Lifecycle	before_stopping	7	Task runner (worker)
Dag Run	on_dag_run_running	0	Scheduler
Dag Run	on_dag_run_success	8	Scheduler
Dag Run	on_dag_run_failed	8	Scheduler

Listeners

Type	Name	Phase	Component
TI	on_task_instance_running	5	Task runner (worker)
TI	on_task_instance_success	7	Task runner (worker)
TI	on_task_instance_failed	7, 8	Task runner (worker), Scheduler
TI	on_task_instance_skipped	7	Task runner (worker)

Cluster Policies

Name	Phase	Component
dag_policy	0	Dag processor
task_policy	0	Dag processor
task_instance_mutation_hook	0, 1	Scheduler
pod_mutation_hook	3, 5	Executor (Scheduler), Task runner (worker)

The background of the slide features a repeating pattern of small, light gray triangles pointing downwards, arranged in a grid-like fashion across the top half of the image.

Stories from the field

ASTRONOMER

Blocking cluster policies

Customer story: Our dags have disappeared.

What happened: Cluster policies made calls to a service which changed its IP and DNS wasn't updated. Retries x timeouts were longer than Dag import timeout.

Takeaways:

- Cluster policies run at the end of the Dag import
- Be mindful of timeouts and retries when making any networking calls
- Handle errors gracefully

Pools at capacity can delay resumption from deferral

Customer story: Our KubernetesPodOperator tasks fail after the pods succeeded.

What happened: Pod cleanup, xcom retrieval and log processing happens after the task has resumed from deferral. Pools were used to control concurrency of tasks and there were many more scheduled tasks than pools slots. Resumed tasks compete with waiting tasks for being queued. The pod had already been deleted by Kubernetes when the tasks managed to resume.

Takeaways:

- Priority weights are materialized when a TI is being scheduled and don't change
- Resumption from deferral can be delayed in systems with tight concurrency control

Priority weights don't help with individual queues at capacity

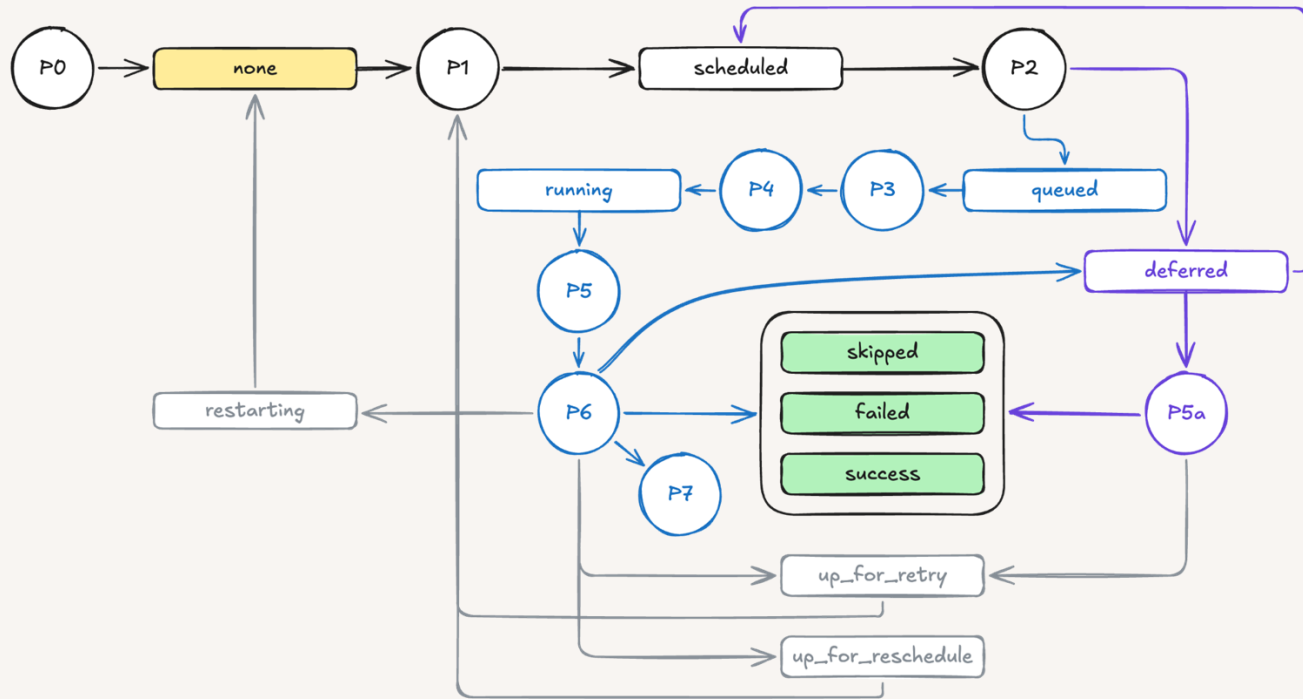
Customer story: High priority tasks were not fast tracked in the queue when a Celery worker was at capacity.

What happened: Priority weights are only considered when queuing tasks in respect to the system's overall capacity. Once a task is queued, it is up to the worker to pick up the task. Celery workers don't consider priority weights when picking up tasks from the queue.

Takeaways:

- Priority weights have no meaning once a task is queued
- Consider pools to reflect queue capacities during queuing decisions
- Also, if one queue is congested, system constraints can starve other empty worker queues

Thank you



The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)

